ML-Based Decision Support System for Smart Investments

Dr. Reshma Koli[1], Avanish Vadke[2], Atharva Shelke[3],
Abhishek Thormothe[4], Aadit Suryawanshi[5],
Prof. Harshada Sonkamble[6]
[123456]A.P. Shah Institute of Technology, Thane, India
rrkoli@apsit.edu.in, avanishvadke3@apsit.edu.in,
atharvashelke2@apsit.edu.in, abhishekthormothe150@apsit.edu.in,
aaditsuryavanshi60@apsit.edu.in, hksonkamble@apsit.edu.in

***Abstract*—Retail investors and discretionary traders make investment decisions under uncertainty, limited information, and behavioral bias. This work presents *QuantFin*, a machine learning–based decision support system for equity investments over Indian large-cap stocks (Nifty 50 constituents available in the repository dataset). The system operationalizes a reproducible workflow: ingestion of daily OHLCV time series from multi-header CSV exports; technical-indicator feature engineering; training and inference across multiple model families (ridge regression, logistic regression, support vector machines (SVM), ARIMA, and long short-term memory (LSTM)); ML-driven ranking for portfolio selection; periodic rebalancing; and leakage-aware walk-forward backtesting against an equal-weight benchmark.**

Quantitative results are reported strictly from repository artifacts and the implemented backtest engine. For example, on the RELIANCE dataset, saved training metrics show test accuracy of 0.5003 (logistic regression) and 0.5136 (SVM) for directional classification, and LSTM test MAPE of 20.447% for price forecasting. In a reproducible walk-forward run (monthly rebalancing, 2023-01-01 to 2024-01-01), the implemented LINEAR strategy achieved CAGR 48.31% versus 30.60% for the benchmark under the repository's backtest assumptions.

***Index Terms*—Decision Support Systems, Quantitative Finance, Portfolio Construction, Walk-Forward Backtesting, Time Series Forecasting, Nifty 50**

I. Introduction

Quantitative investing systems increasingly combine statistical modeling, machine learning, and portfolio construction, but reported performance is sensitive to evaluation protocol: time-order must be respected and realistic constraints (e.g., rebalancing cadence and budget) must be explicit. QuantFin is designed as a full-stack platform that connects these components for the Indian equity universe (Nifty 50 constituents available in the repository dataset). Unlike isolated notebooks, QuantFin operationalizes a repeatable workflow: (i) ingestion of OHLCV data from CSV exports with robust header handling; (ii) technical indicator generation; (iii) model training and prediction orchestration across multiple approaches; (iv) portfolio construction and rebalancing; and (v) walk-forward backtesting with a benchmark.

Problem statement. Retail-facing investment tooling often lacks (a) a transparent, end-to-end data-to-decision pipeline and (b) leakage-aware evaluation that ties predictions to realized returns under a rebalance schedule. This creates a gap between model metrics (accuracy/MAE) and investment outcomes (drawdown, Sharpe).

Objectives. QuantFin aims to provide an end-to-end decision support workflow with clear interfaces (APIs), multiple model baselines, and walk-forward evaluation.

Contributions. This work contributes:

- An end-to-end, modular architecture (FastAPI backend + React/Vite dashboard) for interactive quantitative analysis.
- A leakage-aware walk-forward backtesting procedure that enforces date cutoffs at each rebalance date and reports risk-adjusted metrics.
- A multi-model prediction layer and portfolio construction baselines implemented in the repository.

Organization. Section II reviews related work; Section III formalizes the system model; Section IV describes the proposed methodology; Section V presents architecture and algorithms; Section VI covers implementation details; Section VII defines the experimental setup; and Section VIII reports results and discussion.

II. Related Work

This section links each design motivation to specific post-2020 research, and clarifies which idea is being adopted or contrasted.

End-to-end ML portfolio decision support. Aithal *et al.* [1] describe an IEEE Access real-time portfolio management system that integrates ML prediction with portfolio decisions. QuantFin adopts this end-to-end framing (data $\rightarrow$ model $\rightarrow$ decision $\rightarrow$ evaluation) but implements it as a lightweight FastAPI service with explicit walk-forward evaluation.

Stock ranking and recommendation. Saha *et al.* [2] study stock ranking prediction with list-wise approaches and embeddings. QuantFin similarly uses ranking as the decision interface (select top-N candidates at each rebalance date), but uses a simpler score derived from predicted return and confidence to stay consistent with the repository's implementation.

Classical vs deep time-series baselines. Gao [3] benchmarks ARIMA against deep models for stock prediction.

QuantFin follows this comparative-baseline idea by including ARIMA alongside LSTM, enabling users to compare a classical univariate model to a sequence model in the same pipeline.

LSTM-based trend prediction. Karlis *et al.* [4] apply LSTM architectures for trend prediction by fusing multiple signals. QuantFin includes an LSTM pipeline with windowing and scaling; in the current repository state, the LSTM is used as a forecasting baseline rather than a fully fused multi-source model.

Portfolio optimization baselines. Gunjan and Bhattacharyya [5] provide a Springer review of portfolio optimization methods. QuantFin includes mean–variance and risk-parity style baselines as reference points against the ML-driven ranking strategy, acknowledging that optimization quality depends on covariance estimation.

Dynamic rebalancing and RL approaches. Lim *et al.* [6] and Ma *et al.* [7] present reinforcement-learning based portfolio management and rebalancing. QuantFin does not implement RL, but uses their problem framing to motivate periodic rebalancing and evaluation under realistic sequences of decisions.

Learning market structure. Emekcioğlu and Pınar [8] explore graph neural networks for portfolio optimization. QuantFin currently uses a simpler feature-based approach; the GNN line of work motivates future extensions when cross-asset structure is explicitly modeled.

III. System Model / Problem Formulation

Let $\mathcal{S} = \{1, \ldots, N\}$ be the universe of tradable symbols (Nifty 50 constituents available in the dataset). Let $D = (d_0, d_1, \ldots, d_T)$ be rebalance dates with $d_t < d_{t+1}$. At each rebalance date d_t, the system computes features from data up to d_t and produces a per-asset predicted return $\widehat{r}_{i,t}$ and a confidence score $c_{i,t} \in [0, 1]$.

A. Ranking-based selection

QuantFin's ML-driven ranking uses the score

$$\text{score}_{i,t} = \widehat{r}_{i,t} \cdot c_{i,t} \tag{1}$$

filters assets with $\widehat{r}_{i,t} \leq 0$, selects the top-K assets, and assigns equal weights among selected assets (as implemented in the backtest router):

$$w_{i,t} = \begin{cases} 1/K, & i \in \text{TopK}(\text{score}_{\cdot,t}) \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

The portfolio value evolves by realized returns over each rebalance interval.

B. Benchmark definition

The benchmark is an equal-weighted portfolio of all available symbols, computed over the same rebalance periods. This makes the comparison aligned in both time and rebalancing cadence.

C. Assumptions and constraints

The repository backtest assumes (i) long-only allocation, (ii) discrete rebalancing at fixed cadence, and (iii) no explicit transaction costs or slippage. These constraints are stated to avoid over-claiming.

IV. Methodology / Proposed Method

QuantFin's methodology is a modular pipeline:

1) **Data ingestion:** load per-symbol OHLCV CSV files and normalize headers to a canonical schema.
2) **Feature engineering:** compute return-based and indicator-based features (e.g., SMA/EMA, MACD, RSI, Bollinger bands, rolling volatility).
3) **Model training and inference:** train/serve multiple model families for regression and classification.
4) **Decision support:** convert predictions into rankings/allocations under a capital budget.
5) **Walk-forward evaluation:** simulate rebalancing over time using only information available at each rebalance date.

Given close price P_t, simple returns and log-returns are:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}, \quad \ell_t = \log\left(\frac{P_t}{P_{t-1}}\right) \tag{3}$$

These, along with indicators, form the model input matrix.

A. Feature engineering details (repo-implemented indicators)

QuantFin computes multiple technical indicators directly from OHLCV series. Below we summarize common indicator definitions aligned with the repository implementations.

Moving averages. For window w, the simple moving average (SMA) is

$$\text{SMA}_w(t) = \frac{1}{w} \sum_{k=0}^{w-1} P_{t-k}. \tag{4}$$

The exponential moving average (EMA) uses smoothing factor $\alpha = \frac{2}{w+1}$:

$$\text{EMA}_w(t) = \alpha P_t + (1 - \alpha)\text{EMA}_w(t-1). \tag{5}$$

RSI. Let $\Delta_t = P_t - P_{t-1}$ and define average gains/losses over a window w as G_w and L_w (with $G_w \geq 0$, $L_w \geq 0$). The relative strength is $\text{RS} = \frac{G_w}{L_w}$ and

$$\text{RSI} = 100 - \frac{100}{1 + \text{RS}}. \tag{6}$$

MACD. With fast and slow EMAs, EMA_{12} and EMA_{26},

$$\text{MACD} = \text{EMA}_{12} - \text{EMA}_{26}, \quad \text{Signal} = \text{EMA}_9(\text{MACD}). \tag{7}$$

Bollinger bands. With $m = \text{SMA}_{20}$ and rolling standard deviation σ_{20},

$$\text{Upper} = m + 2\sigma_{20}, \quad \text{Lower} = m - 2\sigma_{20}. \tag{8}$$

TABLE I
REPRESENTATIVE ENGINEERED INDICATORS USED BY QUANTFIN (IMPLEMENTED IN REPOSITORY SERVICES).

Feature family	Examples (typical params)	Purpose / intuition
Returns	r_t, ℓ_t, lagged returns	Captures short-horizon momentum/mean-reversion signals and stabilizes modeling via log-returns.
Trend	SMA/EMA (multiple windows), momentum features	Encodes direction and trend persistence at different time scales.
Oscillators	RSI (windowed gains/losses)	Measures overbought/oversold regimes from relative gain vs loss.
Mean reversion bands	Bollinger bands (20-day mean and variance)	Captures deviation from rolling mean scaled by volatility.
Cross-over signals	MACD and MACD signal	Encodes trend change via fast/slow EMA cross-overs.
Volume	Volume SMA, volume ratio	Provides liquidity/activity proxy and filters price-only signals.

QuantFin Architecture (conceptual).
Data layer: CSV OHLCV $\rightarrow$ preprocessing $\rightarrow$ feature engineering.
Model layer: Linear, Logistic, SVM, ARIMA, and LSTM models with an ensemble option.
Decision layer: ranking/weighting $\rightarrow$ rebalancing $\rightarrow$ backtesting.
Serving layer: FastAPI endpoints consumed by a React dashboard.

Fig. 1. QuantFin system architecture.

V. ALGORITHM / ARCHITECTURE DESIGN

A. System architecture

QuantFin is implemented as a modular full-stack system:

- **Backend:** a FastAPI service exposing endpoints for portfolio rebalancing, analytics (training + metrics), predictions, and walk-forward backtesting.
- **Model services:** reusable components for preprocessing, features, training, inference, and allocation.
- **Frontend:** a React + TypeScript dashboard (Vite) consuming backend APIs.

B. Leakage-aware rebalancing algorithm

Algorithm 1 Leakage-aware monthly rebalancing (simplified)

Input: universe $\mathcal{S}$, rebalance dates D, horizon h, top-K.
Initialize capital V_0.
for $t = 0$ to $|D| - 2$ **do**
  Cutoff date $d \leftarrow D[t]$.
  For each $i \in \mathcal{S}$: compute features using data $\leq d$ and predict $\widehat{r}_{i,t}$.
  Compute confidence $c_{i,t}$ from data quality/volatility/trend fit.
  Rank by $\widehat{r}_{i,t} \cdot c_{i,t}$ and select top-K with $\widehat{r}_{i,t} > 0$.
  Allocate equal weights across selected assets; realize return over $(D[t], D[t+1]]$.
end for
Output: portfolio value series and returns.

VI. IMPLEMENTATION DETAILS

A. Dataset and storage

QuantFin operates on daily OHLCV time series for Nifty 50 constituents stored as CSV files (one file per symbol) under the backend data directory. In the current repository state, the backend data directory contains 49 stock CSV files (excluding auxiliary files such as `nifty50.csv` and `tcs_combined_news.csv`). The global date range across these symbol files spans 1991-01-02 to 2025-10-06.

While an auxiliary news CSV exists at the repository root, it is excluded from the backend's stock-data loader and is not used by the current prediction or allocation flow.

TABLE II
REPOSITORY DATASET SUMMARY (COMPUTED FROM THE BACKEND DATA DIRECTORY).

Property	Value
Number of symbol CSV files	49
Global date range	1991-01-02 to 2025-10-06
Schema after normalization	Date, Open, High, Low, Close, Volume
Frequency	Daily bars (market days)

B. Software stack

The backend is written in Python and served via FastAPI. The implementation uses common scientific Python libraries (NumPy, Pandas, scikit-learn, statsmodels, TensorFlow) and plotting utilities for analysis. The frontend is React + TypeScript built with Vite.

C. API surface (backend)

QuantFin exposes a REST API that maps closely to internal services. Table III summarizes representative endpoint groups used by the dashboard and experiments.

D. Model suite

QuantFin supports complementary model families:

TABLE III
REPRESENTATIVE API ENDPOINT GROUPS IN THE QUANTFIN BACKEND.

Group	Prefix	Typical functionality
Portfolio	/api/portfolio	Summary/performance series, portfolio health, rebalance workflows, cached strategy comparisons.
Analytics	/api/analytics	Training triggers and status, retrieval of cached metrics, candlestick/price-series endpoints backed by local OHLCV.
Predictions	/api/predictions	Per-symbol inference and portfolio recommendations parameterized by model family and prediction horizon.
Backtest	/api/backtest	Walk-forward evaluation via POST /run, returning portfolio/benchmark value series and risk metrics.

- **Ridge regression:** ℓ_2-regularized linear regression

$$\hat{\beta} = \arg\min_{\beta} \|y - X\beta\|_2^2 + \alpha\|\beta\|_2^2 \tag{9}$$

- **Logistic regression:** directional classification

$$\Pr(y = 1 \mid x) = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}} \tag{10}$$

- **SVM:** RBF-kernel classification baseline.
- **ARIMA:** classical statistical baseline for univariate series.
- **LSTM:** sequence model with gated updates

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f), \quad i_t = \sigma(W_i[x_t, h_{t-1}] + b_i), \tag{11}$$

$$\tilde{c}_t = \tanh(W_c[x_t, h_{t-1}] + b_c), \quad c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \tag{12}$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o), \quad h_t = o_t \odot \tanh(c_t) \tag{13}$$

VII. EXPERIMENTAL SETUP

A. Evaluation protocol

QuantFin evaluates strategies using walk-forward backtesting: at each rebalance date, predictions are generated using only historical data up to that date, a portfolio is formed, and realized returns are computed over the next period. This enforces a leakage-aware cutoff by construction.

B. Backtest configuration (repo-grounded)

The example backtest in this paper uses monthly rebalancing from 2023-01-01 to 2024-01-01, initial capital 100000, and top-$K = 10$ selected assets under the LINEAR strategy. The benchmark is computed as an equal-weight basket across all available symbols over the same rebalance dates.

To increase external validity without changing the underlying implementation, we also report two additional configurations produced by the same backtest engine: (i) quarterly rebalancing over the same 2023-01-01 to 2024-01-01 window; and (ii) monthly rebalancing over a longer 2022-01-01 to 2024-01-01 window. These variants change only the evaluation horizon and/or rebalance cadence.

C. Metrics

Given a portfolio value series V_t and periodic returns $r_t = \frac{V_t - V_{t-1}}{V_{t-1}}$, the implemented evaluation reports CAGR, Sharpe ratio, Sortino ratio, maximum drawdown, annualized volatility, Calmar ratio, and win rate.

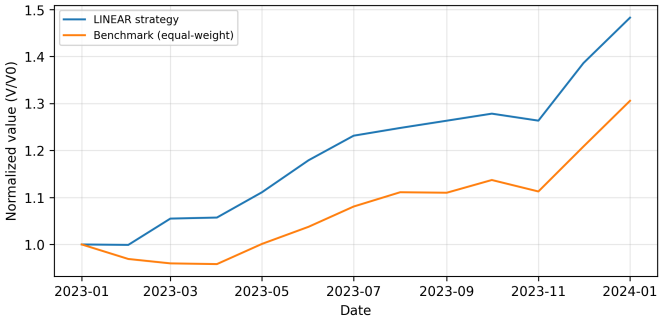

Fig. 2. Normalized equity curve for the implemented LINEAR strategy vs. the equal-weight benchmark (monthly rebalancing, 2023-01-01 to 2024-01-01).

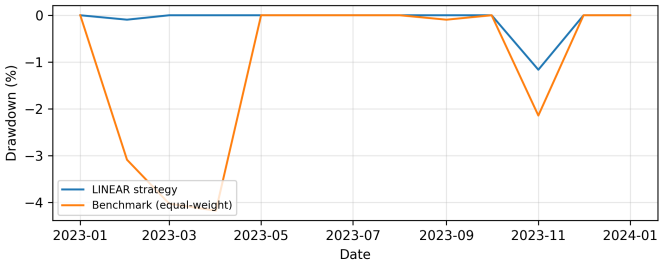

Fig. 3. Drawdown comparison for the implemented LINEAR strategy vs. the equal-weight benchmark over the same walk-forward run.

VIII. RESULTS AND DISCUSSION

A. Sample model metrics

Table IV reports sample metrics produced by QuantFin for the RELIANCE symbol (from an automated training run recorded in the repository). These metrics validate the end-to-end training and evaluation pipeline.

B. Walk-forward backtest example (strategy vs. benchmark)

We ran the repository's walk-forward backtest engine with monthly rebalancing from 2023-01-01 to 2024-01-01 using the implemented LINEAR strategy (top 10, initial capital 100000). Under the repository's assumptions (no transaction costs or slippage), the strategy achieved CAGR 48.31% versus 30.60% for the benchmark, Sharpe 3.04 versus 1.75, and maximum drawdown -1.17% versus -4.18%. Final portfolio value was 148,271 versus 130,578 for the benchmark.

TABLE IV
SAMPLE TRAINING METRICS FOR A REPRESENTATIVE SYMBOL (RELIANCE) FROM A RECORDED QUANTFIN RUN.

Model	Test Acc.	Error (MAE/MAPE)	Notes
Logistic Regression	0.5003	–	Test AUC: 0.5073; test F1: 0.2779
SVM (RBF)	0.5136	–	Best params: $C = 10.0, \gamma = 0.01$; test AUC: 0.5051
Ridge Regression	–	MAE: 0.0742	$\alpha = 100.0$; test $R^2 = -0.3354$
LSTM	–	MAPE: 20.447%	Seq len: 60; units: [128,64]; dropout: 0.2

TABLE V
WALK-FORWARD BACKTEST SUMMARY (MONTHLY REBALANCING, 2023-01-01 TO 2024-01-01).

Portfolio	CAGR	Sharpe	Max DD
LINEAR Strategy	48.31%	3.04	-1.17%
Benchmark (equal-weight)	30.60%	1.75	-4.18%

C. Additional backtest variants

Table VI reports additional repository-generated backtest variants. Quarterly rebalancing yields fewer evaluation periods (four quarters), so metrics that depend on downside samples (e.g., Sortino) can be unstable or undefined; therefore we emphasize CAGR, Sharpe, and maximum drawdown for cross-setting comparison.

D. Discussion

The backtest example demonstrates that the implemented end-to-end workflow can produce measurable differences versus the benchmark under a fixed rebalance cadence. However, the repository's backtest omits explicit transaction costs and slippage, which can materially reduce realized returns. Model metrics also illustrate known challenges: noisy labels, regime shifts, and weak signal-to-noise ratios. Negative test R^2 for the regression baseline indicates that naive baselines can outperform poorly specified targets on some splits.

Across variants, the strategy remains above the benchmark in CAGR for the reported windows. The longer 2022–2024 horizon increases exposure to regime changes and increases drawdown magnitude for both strategy and benchmark, which is consistent with the intuition that longer evaluation windows include more adverse periods. The quarterly cadence reduces turnover by construction, but also reduces the number of decision points, which changes both opportunity and estimation stability.

E. Limitations and future work

Several limitations remain in the current codebase: (i) missing transaction cost and slippage modeling; (ii) simplified covariance assumptions in some optimization baselines (notably in the online prediction service); and (iii) model-selection and hyperparameter choices that may implicitly favor a particular market regime. Future work includes cost-aware simulation, improved label design, and integrating the auxiliary news dataset into the active prediction/portfolio flow once wired end-to-end.

IX. CONCLUSION

This paper presented QuantFin, a machine learning–based decision support system for investments that integrates data preprocessing, technical-indicator feature engineering, a multi-model prediction stack, and leakage-aware walk-forward backtesting over Nifty 50 equities available in the repository dataset. Using repository-generated artifacts, we reported representative model metrics and multiple walk-forward evaluation configurations, including equity-curve and drawdown visualizations comparing the implemented strategy to an equal-weight benchmark.

The results demonstrate that the current implementation is end-to-end functional and can produce measurable differences versus the benchmark under the repository's assumptions. At the same time, the reported performance should be interpreted as decision support rather than guaranteed profit; realistic costs (transaction fees, slippage, liquidity) are not yet modeled and may materially change outcomes. Extending the system to cost-aware evaluation and richer cross-asset and multi-source signals is a concrete next step.

TABLE VI
BACKTEST VARIANTS PRODUCED BY THE REPOSITORY ENGINE (LINEAR STRATEGY VS. EQUAL-WEIGHT BENCHMARK).

Variant	CAGR (S/B)	Sharpe (S/B)	Max DD (S/B)	Final value (S/B)
Monthly (2023-01-01 to 2024-01-01)	48.31/30.60	3.04/1.75	-1.17/-4.18	148,272/130,578
Quarterly (2023-01-01 to 2024-01-01)	40.54/31.09	3.72/2.69	0.00/-4.37	140,503/131,061
Monthly (2022-01-01 to 2024-01-01)	37.47/19.60	1.80/0.90	-5.31/-9.79	188,894/143,007

REFERENCES

[1] P. K. Aithal, M. Geetha, D. U, B. Savitha, and P. Menon, "Real-time portfolio management system utilizing machine learning techniques," *IEEE Access*, vol. 11, pp. 32 595–32 608, 2023.

[2] S. Saha, J. Gao, and R. Gerlach, "Stock ranking prediction using list-wise approach and node embedding technique," *IEEE Access*, vol. 9, pp. 88 981–88 996, 2021.

[3] Z. Gao, "Stock price prediction with arima and deep learning models," in *2021 IEEE 6th International Conference on Big Data Analytics (ICBDA)*, 2021, pp. 61–68.

[4] V. Karlis, K. Lepenioti, A. Bousdekis, and G. Mentzas, "Stock trend prediction by fusing prices and indices with lstm neural networks," in *2021 12th International Conference on Information, Intelligence, Systems & Applications (IISA)*, 2021, pp. 1–7.

[5] A. Gunjan and S. Bhattacharyya, "A brief review of portfolio optimization techniques," *Artificial Intelligence Review*, vol. 56, no. 5, pp. 3847–3886, 2022.

[6] Q. Y. E. Lim, Q. Cao, and C. Quek, "Dynamic portfolio rebalancing through reinforcement learning," *Neural Computing and Applications*, vol. 34, no. 9, pp. 7125–7139, 2022.

[7] C. Ma, J. Zhang, Z. Li, and S. Xu, "Multi-agent deep reinforcement learning algorithm with trend consistency regularization for portfolio management," *Neural Computing and Applications*, vol. 35, no. 9, pp. 6589–6601, 2023.

[8] Ö. Emekcioğlu and M. Ç. Pınar, "Graph neural networks for deep portfolio optimization," *Neural Computing and Applications*, vol. 35, no. 28, pp. 20 663–20 674, 2023.